Case Studies

Senior Data Engineer

Two example case studies about Azure data platforms and production data integrations.

HOW TO READ CASE STUDIES

These case studies go through how to approach common data platform challenges. The idea is to show how approach platform modernization and data integrations from design to running in production.

CASE STUDIES

Modernizing a Legacy Data Platform into an Azure Lakehouse

Moving scattered source systems and old warehouse processes to an Azure lakehouse.

Production Data Integrations

Ingestion and transformation patterns for production: retries, schema changes, data quality and recovering from failures.

MODERNIZING A LEGACY DATA PLATFORM INTO AN AZURE LAKEHOUSE

Representative Scenario

Data is in several operational systems and in an old warehouse. Its hard to modify pipelines and same business logic is implemented in multiple places. Nobody sees the data quality, lineage or why production jobs fail.

Approach

1. List the important sources, data products, owners, access limits and freshness needs. Pick a small but useful part to migrate first.
2. Design a medallion style lakehouse in Azure. Use Databricks and Azure Data Factory where they make sense for the workload.
3. Build ingestion, transformations, data models and automated quality checks with Python, SQL and Spark.
4. Add access control, monitoring, CI/CD and infrastructure automation so releases and environments can be reproduced.
5. Document who owns what, how to recover from failures and what the next migration steps are.

Outcomes to Measure

- How much time and effort it takes to add a new data source.
- Pipeline reliability, data freshness and how fast failures are noticed and fixed.
- Number of data quality issues.
- Running cost and performance.

Relevant Capabilities

Azure Databricks, Azure Data Factory, Apache Spark, Python, SQL, lakehouse architecture, data modeling, data quality, Terraform, CI/CD and production operations.

PRODUCTION DATA INTEGRATIONS

Representative Scenario

Business data comes from APIs, databases and files. The ingestion jobs break easily when a source is down, a schema changes or a run has to be repeated. It is also hard to find out what went wrong when there are failures or data quality problems.

Approach

1. Before implementation, agree on the source contracts, owners, how fresh the data needs to be and how fast recovery is needed.
2. Build ingestion with checkpoints and idempotent processing. Then retries and backfills do not create duplicates or skip data without anyone noticing.
3. Handle schema changes and validate data at the pipeline boundaries so invalid or unexpected records are visible.
4. Add structured logs, metrics and alerts. The operator can then see the impact and recover safely.
5. Automate tests and deployments. Choose full, batch or streaming based on the requirement.

Outcomes to Measure

- Share of successful runs, missing or duplicate records and data freshness.
- Time to detect, find the cause and recover when a source or pipeline fails.
- Effort to add a new source or handle a schema change.

Relevant Capabilities

Azure Data Factory, Azure Databricks, Python, SQL, Apache Spark, API and database integration, data quality, CI/CD, Terraform, monitoring and production operations.